

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

“Stack”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 9 April 2025

M. Fedora Argadyaksa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN.....	3
2.1 Pengertian Stack	3
2.2 Implementasi Stack pada Java	3
2.2.1 Class Nilai Maksimum (NilaiMaksimum_2511533016)	4
2.2.2 Class Siswa Stack (SiswaStack_2511533016)	5
2.2.3 Class Stack Array (StackArray_2511533016)	7
2.2.4 Class Stack Array Driver (StackArrayDriver_2511533016)	8
2.2.5 Class Stack Post fix (StackPostfix_2511533016)	9
BAB III PENUTUP	11
3.1 Kesimpulan.....	11
3.2 Saran	11
DAFTAR PUSTAKA.....	12
LAPORAN TUGAS PRAKTIKUM	13
1. Class Website (Website_2511533016)	13
2. Class Browser (Browser_2511533016)	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, pengelolaan data secara efisien menjadi hal yang sangat penting, terutama ketika berhubungan dengan proses yang bersifat berurutan. Salah satu struktur data yang banyak digunakan untuk menangani kondisi tersebut adalah **Stack**. Stack merupakan struktur data yang bekerja dengan prinsip **LIFO (Last In First Out)**, yaitu elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Konsep ini banyak digunakan dalam berbagai aplikasi seperti manajemen memori, pemanggilan fungsi (call stack), undo-redo pada aplikasi, serta evaluasi ekspresi seperti postfix dan prefix. Oleh karena itu, pemahaman tentang stack sangat penting bagi mahasiswa atau pengembang agar dapat mengimplementasikan solusi yang tepat dalam pemrograman, khususnya dalam menangani data yang membutuhkan pengolahan secara bertumpuk.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 3 tentang Stack adalah sebagai berikut:

1. Memahami konsep dasar struktur data Stack dan prinsip kerjanya (LIFO).
2. Mengetahui operasi dasar pada Stack seperti push, pop, dan peek.
3. Mampu mengimplementasikan Stack menggunakan berbagai cara seperti array maupun ArrayList.
4. Melatih kemampuan logika dan algoritma dalam pemrograman.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 3 tentang Stack adalah sebagai berikut:

1. Membantu dalam mengelola data yang bersifat bertumpuk secara efisien.
2. Mempermudah implementasi fitur penting seperti **undo/redo**, navigasi (back/forward), dan call stack pada program.
3. Digunakan dalam proses parsing dan evaluasi ekspresi dalam compiler atau kalkulator.
4. Meningkatkan pemahaman tentang struktur data lain yang lebih kompleks.
5. Membantu dalam menyelesaikan berbagai permasalahan algoritma yang membutuhkan urutan khusus dalam pengolahan data.

BAB II

PEMBAHASAN

2.1 Pengertian Stack

Stack adalah salah satu struktur data linear yang bekerja berdasarkan prinsip **LIFO (Last In First Out)**, yaitu elemen terakhir yang dimasukkan ke dalam stack akan menjadi elemen pertama yang dikeluarkan. Dalam implementasinya, stack hanya memiliki satu titik akses utama yang disebut **top**, yaitu posisi tempat operasi penambahan (**push**) dan penghapusan (**pop**) dilakukan. Selain itu, terdapat juga operasi **peek** yang digunakan untuk melihat elemen teratas tanpa menghapusnya. Stack dapat diimplementasikan menggunakan array maupun struktur data dinamis seperti linked list atau ArrayList. Struktur data ini banyak digunakan dalam berbagai aplikasi seperti manajemen memori (call stack), evaluasi ekspresi matematika (postfix dan prefix), parsing pada compiler, serta fitur undo dan redo pada perangkat lunak.

2.2 Implementasi Stack pada Java

Pada praktikum pekan 3 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di layar TV. Kode program yang dibuat tentang Array List, di bawah ini kode program yang dituliskan pada praktikum pekan 3.

2.2.1 Class Nilai Maksimum (NilaiMaksimum_2511533016)

```

1 package Pekan3_2511533016;
2
3 import java.util.Stack;
4
5 public class NilaiMaksimum_2511533016 {
6     public static int max_3016(Stack<Integer> s_3016) {
7         Stack<Integer> backup_3016 = new Stack<Integer>();
8         int maxValue_3016 = s_3016.pop();
9         backup_3016.push(maxValue_3016);
10        while (!s_3016.isEmpty()) {
11            int next_3016 = s_3016.pop();
12            backup_3016.push(next_3016);
13            maxValue_3016 = Math.max(maxValue_3016, next_3016);
14        }
15        while (!backup_3016.isEmpty()) {
16            s_3016.push(backup_3016.pop());
17        }
18        return maxValue_3016;
19    }
20    public static void main(String[] args) {
21        Stack<Integer> s_3016 = new Stack<Integer>();
22        s_3016.push(70);
23        s_3016.push(21);
24        s_3016.push(20);
25        System.out.println("isi stack " + s_3016);
26        System.out.println("Stack Teratas " + s_3016.peek());
27        System.out.println("Nilai maksimum " + max_3016(s_3016));
28    }
29 }
30 }
31

```

```

isi stack [70, 21, 20]
Stack Teratas 20
Nilai maksimum 70

```

Gambar 2.2.1

Kode program di atas merupakan program Java yang menggunakan struktur data **Stack** untuk mencari nilai maksimum tanpa menghilangkan isi stack aslinya. Pada bagian awal, package Pekan3_2511533016; menunjukkan lokasi package dari kelas tersebut, dan import java.util.Stack; digunakan untuk mengimpor kelas Stack dari library Java. Kelas NilaiMaksimum_2511533016 berisi method utama max_3016 yang bertipe int dan menerima parameter berupa Stack<Integer> s_3016, dimana fungsi ini bertujuan mencari nilai terbesar dalam stack. Di dalam method ini dibuat stack baru bernama backup_3016 yang berfungsi sebagai tempat penyimpanan sementara agar data stack asli tidak hilang, kemudian elemen pertama diambil menggunakan pop() dan disimpan sebagai nilai awal maksimum (maxValue_3016), lalu dimasukkan ke stack backup. Selanjutnya dilakukan perulangan while selama stack s_3016 belum kosong, setiap elemen di-pop, dimasukkan ke backup, dan dibandingkan dengan nilai maksimum menggunakan Math.max() untuk memperbarui nilai maksimum. Setelah semua

elemen diproses, dilakukan perulangan kedua untuk mengembalikan isi stack dari backup_3016 ke s_3016 sehingga kondisi awal stack tetap terjaga. Method ini kemudian mengembalikan nilai maksimum yang ditemukan. Pada method main, dibuat objek stack s_3016 lalu ditambahkan beberapa nilai menggunakan push() yaitu 70, 21, dan 20, kemudian isi stack ditampilkan, elemen teratas ditampilkan menggunakan peek(), dan terakhir nilai maksimum dari stack ditampilkan dengan memanggil method max_3016(s_3016).

2.2.2 Class Siswa Stack (SiswaStack_2511533016)

```

1 package Pekan3_2511533016;
2
3 import java.util.ArrayList;
4 class Siswa_2511533016 {
5     String nama_3016;
6     int nim_3016;
7
8     public Siswa_2511533016(String nama_3016, int nim_3016) {
9         this.nama_3016 = nama_3016;
10        this.nim_3016 = nim_3016;
11    }
12
13    @Override
14    public String toString() {
15        return "Nim: " + nim_3016 + ", Nama: " + nama_3016;
16    }
17 }
18 public class SiswaStack_2511533016 {
19     private ArrayList<Siswa_2511533016> stack_3016;
20
21     public SiswaStack_2511533016() {
22         stack_3016 = new ArrayList<>();
23     }
24
25     public void push(Siswa_2511533016 mhs_3016) {
26         stack_3016.add(mhs_3016);
27     }
28
29     public Siswa_2511533016 pop() {
30         if (!isEmpty()) {
31             return stack_3016.remove(stack_3016.size() - 1);
32         }
33         return null;
34     }
35
36     public Siswa_2511533016 peek() {
37         if (!isEmpty()) {
38             return stack_3016.get(stack_3016.size() - 1);
39         }
40         return null;
41     }
42
43     public boolean isEmpty() {
44         return stack_3016.isEmpty();
45     }
46
47     public void tampilkanSiswa_2511533016() {
48         for (int i = stack_3016.size() - 1; i >= 0; i--) {
49             System.out.println(stack_3016.get(i));
50         }
51     }
52
53     public static void main(String[] args) {
54         SiswaStack_2511533016 studentStack_3016 = new SiswaStack_2511533016();
55         Siswa_2511533016 mhs1_3016 = new Siswa_2511533016("Ali", 1);
56         Siswa_2511533016 mhs2_3016 = new Siswa_2511533016("Boby", 2);
57         Siswa_2511533016 mhs3_3016 = new Siswa_2511533016("Charles", 3);
58
59         studentStack_3016.push(mhs1_3016);
60         studentStack_3016.push(mhs2_3016);
61         studentStack_3016.push(mhs3_3016);
62
63         System.out.println("Siswa di dalam stack:");
64         studentStack_3016.tampilkanSiswa_2511533016();
65
66         System.out.println("Siswa teratas" + studentStack_3016.peek());
67         System.out.println("Mengeluarkan siswa teratas dari stack: " + studentStack_3016.pop());
68         System.out.println("Mendaftar siswa setelah di pop : ");
69         studentStack_3016.tampilkanSiswa_2511533016();
70     }
71 }
72
73

```

Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
siswa teratasNim: 3, Nama: Charles
Mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
Mendaftar siswa setelah di pop :
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali

Gambar 2.2.2

Kode program pada gambar tersebut mengimplementasikan struktur data **Stack berbasis ArrayList** untuk menyimpan objek mahasiswa. Pada bagian awal terdapat package `Pekan3_2511533016`; yang menunjukkan lokasi package, serta `import java.util.ArrayList`; untuk menggunakan struktur data dinamis `ArrayList`. Kelas `Siswa_2511533016` berfungsi sebagai representasi data mahasiswa dengan atribut `nama_3016` (`String`) dan `nim_3016` (`int`), yang diinisialisasi melalui konstruktor dengan keyword `this` untuk membedakan antara variabel parameter dan atribut kelas. Selain itu, method `toString()` di-override agar ketika objek ditampilkan, output-nya berupa format yang lebih informatif yaitu berisi NIM dan Nama mahasiswa. Selanjutnya terdapat kelas `SiswaStack_2511533016` yang bertindak sebagai implementasi stack menggunakan

`ArrayList<Siswa_2511533016>` bernama `stack_3016`, dimana konstruktor kelas ini menginisialisasi `ArrayList` kosong. Method `push()` digunakan untuk menambahkan objek mahasiswa ke dalam stack dengan cara `add()` ke bagian akhir list, sedangkan method `pop()` digunakan untuk menghapus sekaligus mengembalikan elemen terakhir (`top`) dari stack dengan `remove(size - 1)` setelah memastikan stack tidak kosong menggunakan `isEmpty()`. Method `peek()` berfungsi untuk melihat elemen teratas tanpa menghapusnya dengan `get(size - 1)`, dan method `isEmpty()` digunakan untuk mengecek apakah stack kosong dengan memanfaatkan method bawaan `ArrayList`. Method `tampilkanSiswa_2511533016()` digunakan untuk menampilkan seluruh isi stack dari atas ke bawah dengan perulangan dari indeks terakhir ke indeks pertama sehingga sesuai konsep LIFO. Pada method `main`, dibuat objek stack `studentStack_3016`, kemudian dibuat tiga objek mahasiswa (Ali, Bobby, dan Charles) yang dimasukkan ke dalam stack menggunakan `push()`, lalu program menampilkan seluruh isi stack, menampilkan elemen teratas menggunakan `peek()`, menghapus elemen teratas menggunakan `pop()`, dan akhirnya menampilkan kembali isi stack setelah terjadi perubahan, sehingga memperlihatkan cara kerja stack secara lengkap mulai dari penambahan, pengambilan, hingga penghapusan data.

2.2.3 Class Stack Array (StackArray_2511533016)

```

1 package Pekan3_2511533016;
2
3 public class StackArray_2511533016 {
4     static final int MAX_3016 = 1000;
5     int top_3016;
6     int a_3016[] = new int[MAX_3016];
7     boolean isEmpty_3016()
8     {
9         return (top_3016 < 0);
10    }
11    StackArray_2511533016()
12    {
13        top_3016 = -1;
14    }
15    boolean push(int x_3016)
16    {
17        if (top_3016 >= (MAX_3016 - 1)) {
18            System.out.println("stack Overflow");
19            return false;
20        }
21        else {
22            a_3016[++top_3016] = x_3016;
23            System.out.println(x_3016 + " dimasukkan dalam stack");
24            return true;
25        }
26    }
27    int pop_3016()
28    {
29        if (top_3016 < 0) {
30            System.out.println("Stack Underflow");
31            return 0;
32        }
33        else {
34            int x_3016 = a_3016[top_3016--];
35            return x_3016;
36        }
37    }
38    int peek_3016()
39    {
40        if (top_3016 < 0) {
41            System.out.println("Stack Underflow");
42            return 0;
43        }
44        else {
45            int x_3016 = a_3016[top_3016];
46            return x_3016;
47        }
48    }
49    void print_3016() {
50        for(int i_3016 = top_3016; i_3016 > -1; i_3016--) {
51            System.out.print(" " + a_3016[i_3016]);
52        }
53    }
54    public static void main(String[] args) {
55    }
56    }
57 }
58
59 }
60

```

Gambar 2.2.3

Kode program tersebut merupakan implementasi struktur data **Stack berbasis array** dalam bahasa Java yang dirancang untuk menyimpan data bertipe integer dengan kapasitas maksimum tertentu. Pada bagian awal, package `Pekan3_2511533016`; menunjukkan bahwa kelas ini berada dalam package tertentu, kemudian didefinisikan kelas `StackArray_2511533016` yang memiliki konstanta `MAX_3016 = 1000` sebagai batas maksimal ukuran stack. Variabel `top_3016` digunakan sebagai penunjuk posisi elemen teratas dalam stack, sedangkan array `a_3016[]` berfungsi sebagai media penyimpanan data. Method `isEmpty_3016()` digunakan untuk mengecek apakah stack dalam kondisi kosong dengan cara memeriksa apakah nilai `top_3016` kurang dari 0. Konstruktor `StackArray_2511533016()` berfungsi untuk menginisialisasi nilai `top_3016` menjadi -1 yang menandakan bahwa stack belum berisi elemen. Method `push(int`

x_3016) digunakan untuk menambahkan elemen ke dalam stack, dimana program terlebih dahulu mengecek apakah stack sudah penuh (overflow) dengan membandingkan nilai top_3016 dengan kapasitas maksimum, jika belum penuh maka nilai top_3016 akan dinaikkan terlebih dahulu kemudian elemen dimasukkan ke dalam array dan ditampilkan pesan bahwa data berhasil dimasukkan. Method pop_3016() berfungsi untuk menghapus dan mengambil elemen teratas dari stack, dengan pengecekan kondisi apakah stack kosong (underflow), jika tidak kosong maka elemen pada posisi top_3016 diambil lalu nilai top_3016 dikurangi satu untuk menghapus elemen tersebut dari stack. Method peek_3016() digunakan untuk melihat elemen teratas tanpa menghapusnya, dengan cara mengembalikan nilai pada indeks top_3016 setelah memastikan stack tidak kosong. Method print_3016() digunakan untuk menampilkan seluruh isi stack dari atas ke bawah dengan menggunakan perulangan dari indeks top_3016 hingga 0 sehingga sesuai dengan prinsip LIFO. Terakhir, method main disediakan sebagai titik awal eksekusi program, namun pada kode ini belum diisi sehingga belum ada proses yang dijalankan.

2.2.4 Class Stack Array Driver (StackArrayDriver_2511533016)

```

1 package Pekan3_2511533016;
2
3 public class StackArrayDriver_2511533016 {
4
5     public static void main(String[] args) {
6         StackArray_2511533016 s_3016 = new StackArray_2511533016();
7         s_3016.push(10);
8         s_3016.push(20);
9         s_3016.push(30);
10        System.out.println(s_3016.pop_3016() + " dikeluarkan dari stack");
11        System.out.println("Elemen teratas adalah :" + s_3016.peek_3016());
12        System.out.print("Elemen pada stack : ");
13        s_3016.print_3016();
14    }
15 }
16
17
18 }
19

```

```

10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Elemen pada stack : 20 10

```

Gambar 2.2.4

Kode program tersebut merupakan **kelas driver (program utama)** yang digunakan untuk menguji implementasi stack berbasis array pada kelas StackArray_2511533016. Pada bagian main, pertama dibuat objek stack bernama s_3016 dari kelas StackArray_2511533016, yang berarti stack kosong siap digunakan. Selanjutnya dilakukan beberapa operasi push() yaitu memasukkan nilai 10, 20, dan 30 ke dalam stack secara berurutan, dimana sesuai konsep **LIFO (Last In First Out)** maka elemen terakhir yang masuk (30) akan berada di posisi teratas. Setelah itu, method pop_3016() dipanggil untuk menghapus dan mengambil elemen teratas dari stack, sehingga nilai 30 akan dikeluarkan dan ditampilkan dengan pesan bahwa elemen tersebut telah di-pop dari stack. Kemudian method peek_3016() digunakan untuk melihat elemen teratas yang baru tanpa menghapusnya, yang dalam kondisi ini adalah 20 karena 30 sudah dikeluarkan sebelumnya. Terakhir, method print_3016() dipanggil untuk menampilkan seluruh isi stack yang tersisa, yaitu 20 dan 10, ditampilkan dari atas ke bawah sesuai urutan stack. Secara keseluruhan, kode ini memperlihatkan cara kerja operasi dasar stack yaitu push, pop, peek, dan print dalam satu alur eksekusi program.

2.2.5 Class Stack Post fix (StackPostfix_2511533016)

```

1 package Pekan3_2511533016;
2
3 import java.util.Scanner;
4
5 public class StackPostfix_2511533016 {
6     public static int postfixEvaluate(String expression) {
7         StackInteger s_3016 = new StackInteger();
8         Scanner input_3016 = new Scanner(expression);
9         while (input_3016.hasNext()) {
10             if (input_3016.hasNextInt()) {
11                 s_3016.push(input_3016.nextInt());
12             } else {
13                 String operator_3016 = input_3016.next();
14                 int operand2_3016 = s_3016.pop();
15                 int operand1_3016 = s_3016.pop();
16                 if (operator_3016.equals("+")) {
17                     s_3016.push(operand1_3016 + operand2_3016);
18                 } else if (operator_3016.equals("-")) {
19                     s_3016.push(operand1_3016 - operand2_3016);
20                 } else if (operator_3016.equals("*")) {
21                     s_3016.push(operand1_3016 * operand2_3016);
22                 } else {
23                     s_3016.push(operand1_3016 / operand2_3016);
24                 }
25             }
26         }
27         input_3016.close();
28         return s_3016.pop();
29     }
30     public static void main(String[] args) {
31         System.out.println("Hasil postfix = " + postfixEvaluate("5 2 4 * + 7 -"));
32     }
33 }
34
35

```

Hasil postfix = 6

Gambar 2.2.5

Kode program tersebut merupakan implementasi **evaluasi ekspresi postfix (Reverse Polish Notation)** menggunakan struktur data **Stack** di Java. Pada bagian awal, `package Pekan3_2511533016;` menunjukkan lokasi package, serta `import java.util.Scanner;` dan `import java.util.Stack;` digunakan untuk membaca input string dan memanfaatkan struktur data stack bawaan Java. Kelas `StackPostfix_2511533016` memiliki method utama `postfixEvaluate(String expression)` yang bertipe `int` dan berfungsi untuk menghitung hasil dari ekspresi postfix yang diberikan dalam bentuk string. Di dalam method ini dibuat objek `Stack<Integer> s_3016` sebagai penampung operand (angka), serta objek `Scanner input_3016` yang digunakan untuk membaca token (angka atau operator) dari string ekspresi. Proses utama dilakukan dengan perulangan `while (input_3016.hasNext())`, dimana jika token yang dibaca berupa bilangan (`hasNextInt()`), maka langsung dimasukkan ke dalam stack menggunakan `push()`, sedangkan jika token berupa operator, maka dua elemen teratas diambil dari stack menggunakan `pop()` sebagai `operand2_3016` dan `operand1_3016`, lalu dilakukan operasi sesuai operator (+, -, *, atau /) dan hasilnya dimasukkan kembali ke dalam stack. Proses ini berlanjut hingga seluruh token selesai diproses, kemudian `Scanner` ditutup dan hasil akhir diambil dari stack dengan `pop()` sebagai nilai yang dikembalikan. Pada method `main`, program langsung memanggil `postfixEvaluate("5 2 4 * + 7 -")`, yang berarti menghitung ekspresi postfix tersebut, dan hasilnya ditampilkan ke layar. Secara keseluruhan, kode ini menunjukkan cara kerja evaluasi postfix yang memanfaatkan stack untuk menyimpan operand sementara dan melakukan operasi secara berurutan sesuai aturan postfix.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan pembahasan mengenai struktur data **Stack** dan implementasi kode program yang telah dibuat, dapat disimpulkan bahwa stack merupakan struktur data linear yang bekerja dengan prinsip **LIFO (Last In First Out)**, dimana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Stack memiliki operasi dasar seperti **push, pop, dan peek** yang digunakan untuk mengelola data secara terstruktur melalui satu titik akses yaitu **top**. Dalam implementasinya, stack dapat dibuat menggunakan berbagai metode seperti array maupun struktur data dinamis. Pada kode program yang dibahas, stack diimplementasikan menggunakan array dengan kapasitas tetap, dimana variabel **top** berperan penting dalam menentukan posisi elemen teratas. Program tersebut juga telah mencakup penanganan kondisi **overflow** (stack penuh) dan **underflow** (stack kosong), serta menyediakan fungsi untuk menampilkan isi stack. Secara keseluruhan, baik secara konsep maupun implementasi, stack merupakan struktur data yang sederhana namun sangat penting dan banyak digunakan dalam berbagai aplikasi pemrograman.

3.2 Saran

Adapun saran yang dapat diberikan berdasarkan materi ini adalah sebagai berikut:

1. Kembangkan implementasi stack menggunakan struktur data dinamis seperti Linked List atau ArrayList agar tidak terbatas kapasitas.
2. Tambahkan isi pada method main sebagai contoh penggunaan program agar dapat langsung diuji dan dipahami.
3. Gunakan penamaan variabel yang lebih sederhana dan konsisten supaya kode lebih mudah dibaca.
4. Kembangkan program dengan fitur tambahan seperti pencarian data dalam stack atau penghitungan tertentu.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson dan R. L. Rivest, “Introducing to Algorithms,” 2009. [Online].
- [2] M. T. Goodrich, R. Tamassia dan M. H. Goldwasser, “Data Structures and Algorithms in Java,” 2014. [Online].

LAPORAN TUGAS PRAKTIKUM

SOAL:

Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

1. Class Website (Website_2511533016)

```

1 package Pekan3_2511533016;
2
3 public class Website_2511533016 {
4     String judulWeb_3016;
5     String URL_3016;
6     // Constructor
7     Website_2511533016 (String judulWeb_3016, String URL_3016) {
8         this.judulWeb_3016 = judulWeb_3016;
9         this.URL_3016 = URL_3016;
10    }
11    // Getter
12    public String getjudulWeb_3016() {
13        return judulWeb_3016;
14    }
15    public String getURL_3016() {
16        return URL_3016;
17    }
18    // Setter
19    public void setjudulWeb(String judulWeb_3016) {
20        this.judulWeb_3016 = judulWeb_3016;
21    }
22    public void setURL_3016(String URL_3016) {
23        this.URL_3016 = URL_3016;
24    }
25 }

```

Gambar 1

Program diawali dengan deklarasi package `Pekan3_2511533016;` yang menunjukkan bahwa class ini berada dalam paket tertentu, bertujuan untuk mengorganisasi file agar lebih terstruktur dalam proyek Java. Selanjutnya didefinisikan class `Website_2511533016`, yang berfungsi sebagai blueprint untuk membuat objek website.

Di dalam class terdapat dua atribut (variabel instance), yaitu `judulWeb_3016` bertipe `String` yang digunakan untuk menyimpan judul atau nama website, dan `URL_3016` bertipe `String` yang digunakan untuk menyimpan alamat URL dari website tersebut. Kedua atribut ini tidak diberi modifier akses (default), sehingga hanya dapat diakses dalam package yang sama.

Kemudian terdapat **constructor** `Website_2511533016(String judulWeb_3016, String URL_3016)` yang digunakan untuk menginisialisasi objek saat pertama kali dibuat. Constructor ini menerima dua parameter, yaitu judul website dan URL, lalu mengisi nilai atribut menggunakan keyword `this` untuk membedakan antara variabel parameter dan variabel milik class.

Setelah itu terdapat bagian **getter**, yaitu method yang digunakan untuk mengambil nilai dari atribut tanpa mengubahnya. Method `getjudulWeb_3016()` mengembalikan nilai judul website, sedangkan `getURL_3016()` mengembalikan nilai URL. Getter ini penting dalam konsep enkapsulasi agar akses data tetap terkontrol.

Selanjutnya terdapat bagian **setter**, yaitu method untuk mengubah nilai atribut. Method `setjudulWeb(String judulWeb_3016)` digunakan untuk mengubah judul website, dan `setURL_3016(String URL_3016)` digunakan untuk mengubah URL. Pada kedua setter tersebut, keyword `this` digunakan untuk memastikan bahwa nilai parameter dimasukkan ke dalam atribut class.

Secara keseluruhan, kode ini merupakan contoh sederhana dari penerapan OOP di Java, di mana sebuah class digunakan untuk merepresentasikan data (website) dengan atribut dan method pendukung. Dengan adanya constructor, getter, dan setter, data dapat diinisialisasi, diakses, dan dimodifikasi secara terstruktur serta sesuai prinsip enkapsulasi.

2. Class Browser (Browser_2511533016)

```

1 package Pekan3_2511533016;
2 import java.util.Stack;
3
4 public class Browser_2511533016 {
5     // Tampilan Menu
6     public static void tampilanMenu_2511533016() {
7         System.out.println("===== Browser History KDN: 2511533016 =====");
8         System.out.println("Menu: ");
9         System.out.println("1. Kunjungi Website (Push)");
10        System.out.println("2. Kembali ke Halaman Sebelumnya (Pop)");
11        System.out.println("3. Lihat Halaman Aktif (Peek)");
12        System.out.println("4. Keluar");
13    }
14
15    // Tampilan Website
16    public static void tampilanWebsite(Stack<Website> stackWebsite_2511533016, Scanner sc_3016) {
17        String judulWeb_3016 = sc_3016.nextLine();
18        System.out.println("Masukkan URL : ");
19        String URL_3016 = sc_3016.nextLine();
20        stack_3016.add(new Website_2511533016(judulWeb_3016, URL_3016));
21        System.out.println("Berhasil Menambahkan Website!");
22    }
23
24    // Tampilan Pop
25    public static void tampilanTombolBack(Stack<Website> stackWebsite_2511533016, ArrayList<Website> array_3016) {
26        if (stack_3016.isEmpty()) {
27            System.out.println("History kosong, tidak bisa kembali");
28        } else {
29            Website_2511533016 web_3016 = stack_3016.pop();
30            array_3016.add(web_3016);
31            System.out.println("Kembali dari : " + web_3016.getJudulWeb_3016() + " ke " + web_3016.getURL_3016());
32        }
33    }
34
35    // Tampilan Peek
36    public static void tampilanHalamanAktif(Stack<Website> stackWebsite_2511533016, Scanner sc_3016, ArrayList<Website> array_3016) {
37        if (stack_3016.isEmpty()) {
38            System.out.println("Stack kosong");
39        } else {
40            Website_2511533016 web_3016 = stack_3016.peek();
41            array_3016.add(web_3016);
42            System.out.println("Halaman aktif : ");
43            System.out.println("Judul : " + web_3016.getJudulWeb_3016() + " URL : " + web_3016.getURL_3016());
44        }
45    }
46
47    public static void main(String[] args) {
48        Stack<Website> stackWebsite_2511533016 = new Stack<>();
49        ArrayList<Website> array_3016 = new ArrayList<>();
50        Scanner scanner_3016 = new Scanner(System.in);
51        int choice_3016;
52
53        do {
54            tampilanMenu_2511533016();
55            System.out.println("Pilih menu : ");
56            choice_3016 = scanner_3016.nextInt();
57            scanner_3016.nextLine(); // Consume newline
58
59            switch (choice_3016) {
60                case 1 :
61                    tampilanWebsite(scanner_3016, array_3016);
62                    break;
63                case 2 :
64                    tampilanTombolBack(scanner_3016, array_3016);
65                    break;
66                case 3 :
67                    tampilanHalamanAktif(scanner_3016, array_3016);
68                    break;
69                case 4 :
70                    System.out.println("Keluar dari program.");
71                    break;
72                default :
73                    System.out.println("Pilihan tidak valid.");
74            }
75        } while (choice_3016 != 4);
76        scanner_3016.close();
77    }
78 }
79
80

```

Gambar 2

Program diawali dengan deklarasi package Pekan3_2511533016; untuk mengelompokkan class ke dalam satu paket. Kemudian dilakukan import java.util.Stack untuk menyimpan riwayat halaman web, java.util.Scanner untuk menerima input pengguna, dan java.util.ArrayList yang digunakan sebagai penyimpanan tambahan (misalnya log aktivitas halaman yang di-pop atau diakses).

Class utama Browser_2511533016 berisi beberapa method static yang menangani fitur-fitur browser. Method tampilanMenu_2511533016() digunakan untuk menampilkan menu utama ke layar, yang berisi pilihan seperti mengunjungi website (push), kembali ke halaman sebelumnya (pop), melihat halaman aktif (peek), dan keluar dari program.

Method tampilanWebsite(...) berfungsi untuk menambahkan website ke dalam stack. Program meminta input berupa judul website dan URL dari user melalui Scanner. Data tersebut kemudian dibuat menjadi objek Website_2511533016 dan dimasukkan ke dalam stack menggunakan add() (secara konsep setara dengan push() pada Stack). Setelah itu ditampilkan pesan bahwa website berhasil dikunjungi.

Method tampilanTombolBack(...) merepresentasikan fungsi tombol **Back** pada browser. Method ini pertama-tama mengecek apakah stack kosong menggunakan isEmpty(). Jika kosong, maka ditampilkan pesan bahwa history tidak tersedia. Jika tidak kosong, program mengambil dan menghapus elemen teratas dari stack menggunakan pop(), yang berarti kembali ke halaman sebelumnya. Data website yang di-pop kemudian disimpan ke dalam ArrayList, dan informasi halaman yang ditinggalkan ditampilkan ke layar.

Method tampilanHalamanAktif(...) digunakan untuk melihat halaman yang sedang aktif tanpa menghapusnya dari stack. Jika stack tidak kosong, program mengambil elemen teratas menggunakan peek(), lalu menampilkan judul dan URL halaman tersebut. Data ini juga ditambahkan ke ArrayList. Jika stack kosong, maka ditampilkan pesan bahwa tidak ada halaman sebelumnya.

Pada method main, program mulai dieksekusi dengan membuat objek Stack<Website_2511533016> bernama browser_3016 untuk menyimpan history, serta ArrayList<Website_2511533016> sebagai penyimpanan tambahan. Scanner juga dibuat untuk input user. Variabel choice_3016 digunakan untuk menyimpan pilihan menu.

Program berjalan dalam perulangan do-while sehingga menu akan terus ditampilkan sampai user memilih keluar (opsi 4). Setiap iterasi, program meminta input pilihan menggunakan nextInt() dan membersihkan buffer dengan nextLine(). Kemudian switch-case digunakan untuk menjalankan fitur sesuai pilihan: opsi 1 untuk push (kunjungi website), opsi 2 untuk pop (kembali), opsi 3 untuk peek (lihat halaman aktif), dan opsi 4 untuk keluar. Jika input tidak valid, maka akan muncul pesan kesalahan.

Perulangan berhenti saat user memilih opsi 4, dan Scanner ditutup menggunakan close() untuk menghindari kebocoran resource.

OUTPUT JIKA BERISI :

```
=== Browser History NIM: 2511533016 ===
Menu:
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu : 1
Masukkan Judul : SSO UNAND
Masukkan URL : https://sso.unand.ac.id/login
Berhasil Mengunjungi Website!
=== Browser History NIM: 2511533016 ===
```

```
=== Browser History NIM: 2511533016 ===
Menu:
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu : 2
Kembali dari : SSO UNAND - https://sso.unand.ac.id/login
```

```
=== Browser History NIM: 2511533016 ===
Menu:
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilih menu : 3
Halaman aktif :
Judul : SSO UNAND
URL : https://sso.unand.ac.id/login
```

OUTPUT JIKA KOSONG :

```
=== Browser History NIM: 2511533016 ===
```

```
Menu:
```

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

```
Pilih menu : 2
```

```
History kosong, tidak bisa kembali!
```

```
=== Browser History NIM: 2511533016 ===
```

```
Menu:
```

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

```
Pilih menu : 3
```

```
tidak ada halaman sebelumnya
```